作业书写提示：同学可以直接编辑本文档，在选择题的括号中、简答题的题目之后，用醒目的颜色填写答案；也可以另写一份仅包含答案的文档，但需注意**标明题号**。提交的作业须为 PDF 格式。

Part 1. 进程与线程（50 分）

一、选择题（共 20 分，每题 2 分）

1. 一个进程映像是（ C ）。

   A. 由协处理器执行的一个程序

   B. 一个独立的程序+数据集

   C. PCB 结构与程序和数据的组合

   D. 一个独立的程序

解析：进程映像是进程在某一时刻的组成与状态的集合，通常由程序段、数据段和进程控制块（PCB）构成。程序段描述要执行的操作，数据段保存处理对象和运行数据，PCB 保存进程标识、状态及调度等管理信息。B 缺少 PCB；D 只有程序；A 仅描述程序由何种处理器执行，均不能完整表示进程映像。

2. 进程与程序的根本区别是（ A ）。

   A. 静态和动态特点

   B. 是不是被调入内存

   C. 是不是具有就绪、运行和等待三种状态

   D. 是不是占有处理器

解析：程序是静态的指令集合，进程是程序在某个数据集合上的一次执行过程，具有创建、运行和终止等动态特征。是否调入内存、是否占有处理器都不是根本区别，例如就绪或阻塞的进程并不占有处理器；就绪、运行和等待等状态是进程动态性的具体表现。

3. 在任何时刻，一个进程的状态变化（ C ）引起另一个进程的状态变化。

   A. 必定　　B. 一定不　　C. 不一定　　D. 以上都不是

解析：一个进程的状态变化是否影响其他进程，取决于调度以及进程间的同步关系。例如，某进程因 I/O 完成从阻塞态转为就绪态时，当前运行进程可以继续运行；而运行进程阻塞、调度器选中另一就绪进程时，后者会从就绪态转为运行态。因此既不是必定引起，也不是一定不引起。

4. 进程之间交换数据不能通过（ C ）途径进行。

   A. 管道

B. 消息传递系统

C. 访问目标进程的地址空间

D. 共享内存

解析：每个进程的地址空间是私有的，其他进程不能访问，C 错。共享内存虽然可以由多个进程访问，但每个进程仍然是在访问自己的地址空间。这些地址空间被映射到相同的物理内存上，所以一个进程写入自己的共享内存时，其他进程也能看到各自的共享内存被修改。

5. （ B ）必定伴随进程切换。

A. 一个进程创建后，进入就绪态

B. 一个进程从运行态变为就绪态

C. 一个进程从阻塞态变为就绪态

D. 以上答案都不对

解析：运行进程转为就绪态，说明它因时间片用完或被抢占而让出处理器，操作系统即将调度其他就绪进程运行，因此会发生进程切换。A 中新建进程进入就绪态，C 中阻塞进程被唤醒，都只是使进程具备运行条件，并不表示当前运行进程必须让出处理器，故不一定伴随进程切换。

6. 下列说法中，不正确的是（ C ）。

A. 一个进程可以创建一个或多个线程

B. 一个线程可以创建一个或多个线程

C. 一个线程可以创建一个或多个子进程

D. 一个进程可以创建一个或多个子进程

解析：进程是资源分配和拥有的基本单位，线程是进程内的执行单位。进程可以创建子进程，也可以创建更多线程；线程可以在同一进程中创建其他线程。因此 A、B、D 正确。创建子进程的主体是进程，线程不拥有子进程，故 C 不正确。

7. 【2014 统考真题】一个进程的读磁盘操作完成后，操作系统针对该进程必做的是（ A ）。

A. 修改进程状态为就绪态

B. 降低进程优先级

C. 给进程分配用户内存空间

D. 增加进程时间片大小

解析：进程在发起读磁盘请求后会进入阻塞态，内核完成读操作之后会将进程唤醒，把它设为就绪态；进程能否立即运行还取决于调度。完成一次磁盘读取并不要求降低优先级、重新分配用户内存、增大时间片，故 B、C、D 均不是必做操作。

注：题目中的“读磁盘操作”或类似表述都大致可看作 Linux 的 `read()` 系统调用（这个系统调用是同步 I/O），而且假定确实要从磁盘读数据，进程发起请求后通常会阻塞，完成后会进入就绪态。事实上，如果数据在页缓存中，或者发起的是异步 I/O 请求，是不会阻塞的，也就没有进入就绪态一说。

8. 【2014 统考真题】下列关于管道（pipe）通信的叙述中，正确的是（ C ）。

A. 一个管道可实现双向数据传输

B. 管道的容量仅受磁盘容量大小限制

C. 进程对管道进行读操作和写操作都可能被阻塞

D. 一个管道只能有一个读进程或一个写进程对其操作

解析：在通常的阻塞式管道通信中，管道为空且仍有写端打开时，读进程可能阻塞；管道缓冲区已满时，写进程也可能阻塞，直到出现可用空间。因此读、写操作都有可能被阻塞。

A 错：一个管道提供单向数据传输，若要实现双向通信，需要建立两条管道。B 错：管道通常依靠内存中的缓冲区传递数据，与磁盘容量基本无关。D 错：一个管道可以由多个进程访问，并不限定只能有一个读进程或一个写进程；多个进程共同使用时，通常应通过同步与互斥保证操作有序。

9. 在多对一的线程模型中，当一个多线程进程中的某个线程被阻塞后，(B)。

A. 该进程的其他线程仍可继续运行。

B. 整个进程都将阻塞

C. 该阻塞线程将被撤销

D. 该阻塞线程将永远不可能再执行

解析：在多对一的线程模型中，由于只有一个内核级线程，用户级线程的“多”对操作系统透明，因此内核只能感知到一个调度单位的存在。因此，如果该进程的一个用户级线程被阻塞，该内核级线程（即该进程）就被阻塞，其他用户级线程也就都被阻塞。

10. 下面关于用户级线程和内核级线程的描述中，错误的是（ B ）。

A. 多处理器系统可以调度同一进程中的多个内核级线程并行运行。

B. 跨进程调度用户级线程不需要内核参与，控制简单

C. 大多数通用操作系统通常允许用户程序内部实现和调度用户级线程

D. 若系统不支持一个进程内有多个内核级线程，则系统的调度单位是进程

解析：内核级线程是操作系统对用户程序的调度单位，进程是操作系统对用户程序的资源分配单位，对二者的管理都需要操作系统内核参与（如果不支持内核级线程，则合二为一）。

在跨进程调度用户级线程时，暂停运行和开始运行的两个用户级线程分属两个进程中各自的一个内核级线程，因此实际上发生了内核级线程调度，需要内核参与，B 错。内核级线程是调度单位，多个内核级线程可以被同时分配到多个处理器上运行，A 对。用户级线程几乎不依赖于特定操作系统，操作系统也无法干预用户级线程的调度，C 对。如果系统不支持内核级线程，则进程既是资源分配单位，又是调度单位，D 对。

二、简答题（共 30 分）

1. 在采用优先级进程调度时，运行中的进程是否一定包括系统中优先级最高的进程？为什么？（4 分）

答：不一定。因为高优先级的进程可能处于阻塞状态，被运行的非阻塞进程可能优先级较低。或：采用的可能是非抢占式调度。或：即便是抢占式调度，也未必保证高优先级进程在运行。等等，言之成理即可。

2. 某单处理机系统只考虑运行态、就绪态、阻塞态。进程 P 初始处于运行态，随后依次发生以下事件：P 请求读取磁盘，但所需数据尚未准备好；磁盘读取完成；调度程序选中 P 占用 CPU；P 的时间片用完，但尚未执行结束。请按事件发生顺序直接写出进程 P 的完整状态变化序列，并说明每次状态转换的直接原因。（10 分）

答：

运行态→阻塞态→就绪态→运行态→就绪态。

P 请求读取尚未准备好的磁盘数据时，需要等待 I/O 完成，因此由运行态转为阻塞态。磁盘读取完成后，P 已具备运行条件但尚未获得 CPU，因此由阻塞态转为就绪态。调度程序选中 P 后，P 获得 CPU，由就绪态转为运行态。最后，P 的时间片用完且尚未结束，操作系统抢占式收回 CPU，P 由运行态转为就绪态。

3. fork()进程创建与输出分析。假设 fork()不会失败，printf()是原子的。阅读以下 C 程序：

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int x = 1;
    pid_t pid;

    pid = fork();            // F1
    if (pid == 0) {
        x += 1;
        if (fork() == 0) {    // F2
            x += 1;
            printf("A: x=%d\n", x);
        } else {
            printf("B: x=%d\n", x);
        }
    } else {
        x += 10;
        printf("C: x=%d\n", x);
    }
    printf("D: x=%d\n", x);
    return 0;
}
```

（1）直接写出程序一共创建了几个进程（包括初始进程）。然后，给出除初始进程 P0 之外的每个进程被哪个进程在哪一行 fork()调用中创建。约定用 P1、P2 等标号指代除 P0 之外的进程；用程序注释中的 F1、F2 指代 fork()调用。（2 分）

（2）直接写出程序的一种可能输出，要求完整写出每一行输出。然后回答：父子进程中对 x 的修改会互相影响吗？为什么？（5 分）

（3）假设在语句“x += 10”之前，紧邻该语句，插入语句“wait(NULL);”。“wait(NULL);”的作用是什么？它对输出顺序有什么影响？要求在回答中用你在（1）中的标号明确指出该语句所涉及的进程，并直接写出输出中哪些行之间的顺序关系是可以确定的，哪些是不能确定的，可以确定的顺序是什么。（5 分）

（4）用 sleep()调整该程序中不同进程之间的输出关系，使一个进程的输出在另一进程之后，是绝对可靠的吗？为什么？（sleep()的作用是：将调用它的进程设置为睡眠态，持续指定时间后才可继续执行。假设睡眠中的进程不会被意外唤醒。）（4 分）

答：

（1）程序一共创建了 3 个进程。P0 通过 F1 创建子进程 P1，P1 通过 F2 创建子进程 P2。

（2）P0 输出：“C: x=11”“D: x=11”。P1 输出：“B: x=2”“D: x=2”。P2 输出：“A: x=3”“D: x=3”。进程之间的输出没有顺序关系，每个进程内部须确保“A/B/C”先输出，“D”后输出。任意给出一种正确的输出顺序即可。一种正确答案如下：

C: x=11

B: x=2

A: x=3

D: x=3

D: x=2

D: x=11

父子进程中对 x 的修改不会相互影响。因为在 fork()后，子进程获得父进程地址空间的一份副本，所以各自拥有一个变量 x，进程对自己的 x 的修改不会影响到其他进程地址空间中的 x。

（3）“wait(NULL);”的作用是：P0 阻塞等待其子进程 P1 结束，并回收该子进程，避免产生不被回收的僵尸进程。（回答“P0 等待 P2”“P0 等待 P1

和 P2”的，此处不得分。）

可以确定两组顺序：① B: x=2 → D: x=2 → C: x=11 → D: x=11；② A: x=3 → D: x=3。不能确定这两组输出之间的顺序关系。

（4）不可靠。因为 `sleep()`只保证本进程暂停执行一段时间，不保证其他进程在这段时间内被内核调度并执行输出语句。在这种情况下，不同进程的输出顺序仍然取决于内核的调度，而内核的调度行为通常是不可依赖的。

Part 2. 进程与线程（50 分）

一、选择题（共 14 分，每题 2 分）

1. 进程 P 正在执行临界区代码时因等待磁盘 I/O 而阻塞。对于与该临界资源对应的同一个临界区，下列说法正确的是（ B ）。

A. 另一个进程可以立即进入该临界区，因为 P 已经不占用 CPU

B. 另一个进程不能进入该临界区，但可以被调度执行其临界区之外的代码

C. 任何进程都不能被调度，直到 P 的 I/O 完成

D. 另一个进程可以进入该临界区，但不能修改临界资源

解析：P 阻塞后不能继续占有 CPU，但它仍处在临界区的执行过程中，不允许其他进程进入同一个临界区。其他进程可以运行非临界区代码。

2. 计数信号量 S 采用如下约定：$S > 0$ 时 S 表示可立即获得的资源数，$S = 0$ 表示无可用资源且无等待者，$S < 0$ 时 $-S$ 表示等待进程数。若当前 $S = 1$，设 M 为可立即获得的资源数、N 为阻塞等待的进程数，则（ B ）。

A. $M = 0$，$N = 1$

B. $M = 1$，$N = 0$

C. $M = 1$，$N = 2$

D. $M = 2$，$N = 0$

解析：题目已经说明 $S > 0$ 时 S 就是可用资源数，且没有等待者。因此当前 $S = 1$ 时 $M = 1$、$N = 0$。

3. 两个进程使用初值为 `1` 的互斥信号量 `mutex` 保护同一临界区。若 `P(mutex)` 不是原子操作，最直接的风险是（ A ）。

A. 两个进程都可能读到 `mutex = 1`，并都将其改为 `0` 后进入临界区

B. `mutex` 一定会变成负数，因此两个进程都被阻塞

C. `V(mutex)`无法把阻塞进程唤醒

D. 以上都不是

解析：若在判断 $S > 0$、修改 S 两步之间可被切换，两个进程都可能先读到 1，再分别写入 0，从而同时通过检查并进入临界区，互斥性失效。

4. 有 4 个并发进程共享一段程序，每次最多允许 3 个进程同时进入。用计数信号量 S（初值为 3）实现限制，且 `P(S)`在 $S = 0$ 时使进程阻塞，则 S 在整个运行

过程中可能取到的所有整数值是（ B ）。

A. 4，3，2，1，0

B. 3，2，1，0，−1

C. 2，1，0，−1，−2

D. 3，2，1，0，−1，−2

解析：S 从 3 开始，最多允许 3 个进程占用。前三次 P 使 S 依次为 2、1、0；第四个进程执行 P 时阻塞，按题设语义 S 变为 −1。

5. 一个有界缓冲区有 N 个空槽位，使用信号量 `empty = N`、`full = 0`、`mutex = 1`。生产者每次放入一个数据，且放入操作必须在 `mutex` 保护下完成。生产者放入一个数据时，以下操作顺序正确的是（ A ）。

A. `P(empty)`；`P(mutex)`；放入数据；`V(mutex)`；`V(full)`

B. `P(mutex)`；`P(empty)`；放入数据；`V(full)`；`V(mutex)`

C. `P(empty)`；放入数据；`P(mutex)`；`V(mutex)`；`V(full)`

D. `P(full)`；`P(mutex)`；放入数据；`V(mutex)`；`V(empty)`

解析：生产者必须先 `P(empty)`确认有空槽，再 `P(mutex)`进入临界区放入数据；放入完成后先 `V(mutex)`释放临界区，再 `V(full)`通知消费者。如果按照 B 选项那样，由 `P(mutex)`和 `V(mutex)`包裹的整个内部都变成互斥的，如果一个生产者在 `P(empty)`时发现无空槽位，就会永远阻塞在 `empty` 上。

6. 在抽象并发模型中，共享整数 `count` 初值为 `0`。两个进程各执行一次 `count++`；每次 `count++`明确分为“读取 `count`”和“将读取值加 `1` 后写回 `count`”两步，且没有任何同步。假设每次单独的读或写不可再分，则最终 `count` 可能为（ D ）。

A. 只能为 `0`

B. 只能为 `1`

C. 只能为 `2`

D. 为 `1` 或 `2`

解析：两个进程可能先后读取到同一个旧值 0，再分别写回 1，因而最终为 1；若一个进程完成写回 1 后另一个才读取，则最终为 2。抽象模型已明确读、写均不可再分，因此不可能为 0。

7. 下列选项中（ D ）不是产生死锁的必要条件。

A. 互斥使用资源

B. 占有并等待

C. 不可剥夺

D. 进程按固定顺序轮流获得 CPU 时间片

解析：死锁的四个必要条件是互斥、占有并等待、不可剥夺和循环等待。CPU 时间片轮流分配不是死锁的必要条件。

二、简答题（共 36 分）

1. 一个生产者 PA 和一个消费者 PC 共享有 N 个槽位的有界缓冲区（N > 0）。PA 每次生产 1 个数据并放入缓冲区，PC 每次从缓冲区取出 1 个数据；放入和取出操作必须互斥，缓冲区不能溢出或下溢。请用信号量 `empty`、`full`、`mutex` 和 P、V 操作实现 PA、PC 对缓冲区的互斥访问。写出这些信号量在你的实现中的初值和含义，然后给出 PA、PC **循环**写入或读取缓冲区的伪代码。

约定：信号量 `empty` 表示空槽数，`full` 表示已有数据数，`mutex` 用于保护缓冲区操作。用描述性名称、中文、注释等方式使伪代码易读。（9 分）

答：

```
semaphore empty = N;   // 空槽位数
semaphore full = 0;    // 已有数据数
semaphore mutex = 1;   // 缓冲区互斥锁

PA:
while (true) {
    生产一个数据;
    P(empty);
    P(mutex);
    将数据放入缓冲区;
    V(mutex);
    V(full);
}

PC:
while (true) {
    P(full);
    P(mutex);
```

```
    从缓冲区取出一个数据;
    V(mutex);
    V(empty);
    消费该数据;
}
```

2. 理发店里有一位理发师、一把理发椅和 n 把供等候顾客坐的椅子。若没有顾客，理发师等待（阻塞）；有顾客到来时，理发师为他理发。若理发师正在理发时又有顾客来到，若有空椅子可坐，则顾客坐下来等待（阻塞），否则离开（退出函数）。用信号量和 P、V 操作实现理发师循环服务顾客和单个顾客接受服务（或等待、离开）的行为，提供伪代码，并在伪代码之前说明所用信号量和辅助变量的定义和初值。

约定：在理发师要理发、顾客要接受理发时，理发师调用一次 `cut_hair()`，顾客调用一次 `get_haircut()`。认定有一方调用函数后理发开始，双方调用均返回时理发结束。如果 `get_haircut()`被并发调用，认定程序出错。如果一次 `cut_hair()`调用与多次 `get_haircut()`调用在时间上重叠，认定程序出错；反之亦然。你在伪代码中定义的全局变量默认是在进程间共享的。

提示：在你的伪代码中，理发师的实现须形如：

```
barber() {
    while (true) {
        ...
        cut_hair();
        ...
    }
}
```

顾客的实现须形如：

```
customer() {
    ...
    get_haircut();
    ...
}
```

`barber()`只被调用一次，永不返回；`customer()`可被并发调用。（15 分）

参考答案：

1. 信号量 `customers` 表示等待人数，初值为 `0`。

2. 信号量 start 表示可开始理发（理发师空闲），初值为 0。

3. 信号量 finished 表示一名顾客的理发刚刚结束，初值为 0。

4. 整型变量 waiting 表示等待人数，初值为 0。（与 customers 的区别在于，waiting 在 customer()中用于检查大小。）

5. 信号量 mutex 互斥保护对 waiting 的访问，初值为 1。

```
int waiting = 0;
semaphore start = 0, finished = 0, mutex = 1;

barber() {
    while (true) {
        P(customers);
        V(start);
        cut_hair();
        P(finished);
    }
}
customer() {
    P(mutex);
    if (waiting < n) {
        waiting = waiting + 1;
        V(mutex);
        V(customers);
        P(start);
        P(mutex);
        waiting = waiting - 1;
        V(mutex);
        get_haircut();
        V(finished);
    } else {
        V(mutex);
    }
}
```

说明：答案不唯一。参考答案的并发保护由维护最大等待人数的 waiting 和 mutex，以及保证理发并发正确性的 start 和 finished 两部分构成。为维护最大等待人数，一个用于互斥的信号量和一个用于计数的普通变量是必须的，只使用信号量必定是错误答案（除非把信号量当作普通变量使用）。为保证并发正确，必须确保理发师和顾客的理发临界区在时间上是包含的：这边进入临界区后，那边才进入临界区；那边退出临界区后，这边才退出临界区。不满足此条件的均为错误答案。

此外，简单允许 n + 1 个顾客理发或等待（从而在有 1 个人理发时有 n 个人等待）的实现也可以接受，虽然在这种实现下有可能短暂发生有 n + 1 个顾客进程同时阻塞在同一个信号量上的情况。

3. 下表给出了整型信号量 S 的 wait()和 signal()操作的功能描述，以及采用开/关中断指令在**单核**机器上实现信号量操作互斥的两种方法。

功能描述	方法 1	方法 2

Semaphore S;
 	
wait(S) {
  while (S <= 0);
  S = S - 1;
}
 	
signal(S) {
  S = S + 1
}
	Semaphore S;
	
	wait(S) {
	  关中断;
	  while (S <= 0);
	  S = S - 1;
	  开中断;
	}
	
	signal(S) {
	  关中断;
	  S = S + 1
	  开中断;
	}
		Semaphore S;
		
		wait(S) {
		  关中断;
		  while (S <= 0) {
		  开中断;
		  关中断;
		  }
		  S = S - 1;
		  开中断;
		}
		
		signal(S) {
		  关中断;
		  S = S + 1
		  开中断;
		}

请回答下列问题：

（1）为什么在 wait()和 signal()操作中对信号量 S 的访问必须互斥执行？（2 分）

（2）分别说明方法 1 和方法 2 是否正确，并说明理由。（6 分）

（3）用户程序能否使用开/关中断指令实现临界区互斥？为什么？（4 分）

答：

（1）信号量 S 在进程间共享，多个进程都可通过 wait()和 signal()对 S 进行读写操作，如果访问不互斥会发生竞争（或错误修改、不一致等）。

（2）方法 1 错误。在 wait()中，当 S ≤ 0 时，关中断后，其他进程无法修改 S 的值，while 语句陷入死循环。方法 2 正确。方法 2 在循环体中有一个开中断操作，这样就可以调度到其他进程，S 仍有机会被修改。（注：在单核机器

上，关中断后，内核无法调度到其他进程；开中断后，如果硬件发现有待处理的中断〔例如定时触发的时钟中断〕，CPU 会立刻保存执行上下文并跳转到内核中的中断处理程序，接着内核可以调度到其他程序继续执行。）

（3）不行。因为开/关中断指令是特权指令，不能在用户态中执行，只能在内核态中执行。

注：在实际的单核机器上，内核对一个信号量忙等待是没有意义的，通常会在发现 `wait()`操作须阻塞时设置当前进程为睡眠状态，将其加入 S 的等待队列，跳转到调度器。

Part 3. 附加题（10 分）

加分规则：附加题分数与第一次作业分数相加，即本次作业总分多 10 分，并计入课程四次作业总分，但四次作业总分不超过课程考核标准中规定的分数占比。

为助力国产操作系统生态建设，本课程组联合华为设置以下**选做任务**，同学们自愿参与，完成任务可获得平时作业额外加分。

任务：HarmonyOS 应用开发者基础认证

同学们完成学习并通过认证考试，提交合格证书即可申请加分。认证考试不超过 60 分钟。认证入口：

https://developer.huawei.com/consumer/cn/training/dev-cert-detail/101666948302721398

合格证书样例：

